# RHCSA (EX200) Study Guide RHEL 9

> Reorganized from personal notes, corrected, and completed with the missing exam objectives. Sections tagged ➕ **Added** were not in the original notes. Items tagged 🖍️ **Fixed** corrected an error in the originals.

## Table of Contents


1. Essential Tools & Shell
2. Users, Groups & Sudo
3. Permissions, umask & ACLs
4. Processes, Tuning & Services
5. Boot, GRUB & Recovery
6. Software Management
7. Scheduling Tasks
8. Networking & SSH
9. Firewall
10. SELinux
11. Storage: Partitions, Swap, fstab
12. LVM & VDO
13. NFS & autofs
14. Time Sync (chrony)
15. Logging & journald
16. Containers (Podman)
17. Shell Scripting
18. Exam Tips & Final Checklist


---

## 1. Essential Tools & Shell

> ➕ **Added**: this whole section.

### Getting help

```
man -k keyword          # search man pages (run mandb first if empty)
man 5 fstab             # section 5 = config file formats
command --help
ls /usr/share/doc/
```

### I/O redirection & pipes

```
cmd > file      # overwrite stdout
cmd >> file     # append stdout
cmd 2> err.txt  # stderr only
cmd &> all.txt  # stdout + stderr
cmd1 | cmd2     # pipe
cmd | tee file  # display and save
```

## Text processing

```
grep -rniE "pattern" /path     # recursive, line numbers, ignore case, regex
grep -v pattern file           # invert match
cut -d: -f1,3 /etc/passwd
sort | uniq -c | sort -nr
wc -l file
head -n 5 file ; tail -f file
sed -i 's/old/new/g' file
awk -F: '{print $1,$3}' /etc/passwd
```

## Files & directories

```
cp -a src dst        # preserve everything
mv a b
rm -rf dir
mkdir -p a/b/c
ln file hard         # hard link (same inode, same filesystem)
ln -s target soft    # symbolic link
```

## Finding files

```
find / -name "*.conf" -type f
find /tmp -type f -mtime +3 -exec rm -f {} \;      # older than 3 days; runs
rm once per file
find "$1" -type f -size +1M -exec ls -lhS {} +     # `+` batches all files
into ONE ls call
find / -user aymane                                # files owned by a user
find / -perm -4000 2>/dev/null                     # SUID files
locate file ; which cmd ; whereis cmd
```

## Archiving & compression

```
tar -czvf backup.tar.gz /dir    # create gzip
tar -cjvf backup.tar.bz2 /dir   # create bzip2
tar -cJvf backup.tar.xz /dir    # create xz
```

```
tar -xvf backup.tar.gz -C /dest # extract (auto-detects compression)
tar -tvf backup.tar.gz          # list contents
```

### Remote copy

```
scp file aymane@192.168.100.189:/home/aymane          # local → remote
scp aymane@192.168.100.189:/home/aymane/client_file /root   # remote → local
rsync -avz /src/ user@host:/dest/
```

### vim survival

`i` insert · `Esc` · `:wq` save/quit · `:q!` quit without saving · `dd` delete line · `yy` / `p` copy/paste · `/text` search · `:%s/old/new/g` replace.

---

## 2. Users, Groups & Sudo

### Users & groups (➕ Added)

```
useradd -m -s /bin/bash -G wheel -u 2000 aymane
useradd -s /sbin/nologin svcuser          # no interactive login
usermod -aG groupname aymane              # ALWAYS use -a with -G (append)
usermod -L aymane ; usermod -U aymane     # lock / unlock
userdel -r aymane                         # delete + home dir
groupadd -g 3000 devs
id aymane ; groups aymane
echo "NewPassword" | passwd --stdin aymane   # set password in a script
passwd -S aymane                          # status (LK = locked)
```

Key files: `/etc/passwd`, `/etc/shadow`, `/etc/group`, `/etc/skel` (template copied into new home dirs).

### Password policy

**Complexity**: `/etc/security/pwquality.conf`

```
minlen = 12
minclass = 3
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
retry = 3
```

**Aging defaults for new users**: `/etc/login.defs`

```
PASS_MAX_DAYS 90
PASS_MIN_DAYS 7
PASS_WARN_AGE 14
PASS_MIN_LEN 8
```

**Aging for an existing user**: `chage`

```
chage -l aymane                       # show settings
chage -M 90 -m 7 -W 14 aymane         # max, min, warn days
chage -E 2026-12-31 aymane            # account expiry date
chage -d 0 aymane                     # force password change at next login
```

**Lockout after failed logins**: `/etc/security/faillock.conf`

```
deny = 5
unlock_time = 600
```

```
faillock --user aymane            # view failures
faillock --user aymane --reset    # unlock
```

## sudo

```
visudo                          # safe editing of /etc/sudoers
visudo -cf /etc/sudoers.d/aymane   # check syntax of a file
```

**Option A**: drop-in file (preferred):

```
echo 'aymane ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/aymane
chmod 0440 /etc/sudoers.d/aymane
visudo -cf /etc/sudoers.d/aymane
```

**Option B**: edit `/etc/sudoers` and add under `root`:

```
aymane   ALL=(ALL)  ALL
%wheel   ALL=(ALL)  ALL        # whole group
```

## Per-user login message / environment

- Message at login: add the script/command at the **bottom** of `/home/user/.bash_profile`, or create `/usr/local/bin/message`, `chmod +x` it, and call it from `.bash_profile`.
- Per-user umask → `~/.bash_profile` (see §3).

# 3. Permissions, umask & ACLs

## Basic permissions

```
chmod 755 file ; chmod u+x,g-w,o=r file
chown user:group file ; chown -R user: dir
```

| Octal | Meaning |
|---|---|
| 4 | read |
| 2 | write |
| 1 | execute |

## Special permissions

| Bit | Octal | On… | Effect |
|---|---|---|---|
| SUID | 4000 | file | runs as the file **owner** |
| SGID | 2000 | file / dir | runs as group / **new files inherit the dir's group** |
| Sticky | 1000 | dir | only owner can delete their files (e.g. `/tmp`) |

```
chmod g+s /shared          # or chmod 2770 /shared
chmod +t /shared           # or chmod 1777 /shared
```

> ⚠️ **Order matters**: change owner/group **first** (`chown`), **then** apply `chmod`. `chown` can strip SUID/SGID.

## umask

Default permissions = `666` (files) / `777` (dirs) minus the umask.

| Scope | Where | Example |
|---|---|---|
| System-wide | append to `/etc/profile` (survives reboots) | `umask 0022` |
| One user | `/home/user/.bash_profile` (overrides system) | `umask 0027` |

> 🧨 **Fixed**: the original note had `umask 4624`, which is invalid. A umask is 3-4 octal digits (e.g. `0027`, `0077`). Your `0277` example works but is unusual.

Apply immediately: `source ~/.bash_profile` or `exec bash`.

### ACLs

```
setfacl -m u:user:rwx /path        # grant user
setfacl -m g:group:r-x /path       # grant group
setfacl -x u:user /path            # remove one entry
setfacl -b /path                   # remove all ACLs
setfacl -d -m u:user:rwx /dir      # DEFAULT ACL (inherited by new files)
setfacl -R -m u:user:rx /dir       # recursive
getfacl /path
```

A `+` at the end of `ls -l` permissions means an ACL is set.

---

## 4. Processes, Tuning & Services

> ➕ **Added**: process control and systemd basics.

### Processes

```
ps aux ; ps -ef --forest
top ; htop
kill PID ; kill -9 PID ; pkill name ; killall name
nice -n 10 cmd             # start with lower priority (-20 high … 19 low)
renice -n 5 -p PID
jobs ; bg ; fg ; cmd &
```

### tuned profiles

```
dnf install -y tuned
systemctl enable --now tuned
tuned-adm list
tuned-adm active
tuned-adm recommend
tuned-adm profile throughput-performance
```

### systemd services

```
systemctl status|start|stop|restart|reload sshd
systemctl enable --now sshd         # start now + at boot
systemctl disable --now sshd
systemctl mask|unmask sshd          # fully block
systemctl is-active|is-enabled sshd
systemctl list-units --failed
systemctl list-unit-files --type=service
```

### Targets

```
systemctl get-default
systemctl set-default multi-user.target     # or graphical.target
systemctl isolate rescue.target
```

### Kernel parameters

```
sysctl -a | grep ip_forward
echo "net.ipv4.ip_forward = 1" > /etc/sysctl.d/99-forward.conf
sysctl --system
```

---

## 5. Boot, GRUB & Recovery

### GRUB configuration

Edit `/etc/default/grub` (🖍️ **Fixed**: the path is `default`, not `defaults`), then regenerate:

```
grub2-mkconfig -o /boot/grub2/grub.cfg          # BIOS
grub2-mkconfig -o /boot/grub2/grub.cfg          # UEFI on RHEL 9 also uses
this path
```

### Reset the root password (➕ **Added**: very common exam task)

1. Reboot; at the GRUB menu press `e` on the kernel entry.
2. On the line starting with `linux`, append: `rd.break`
3. Press **Ctrl+X** to boot.
4. At the `switch_root:/#` prompt:

```
mount -o remount,rw /sysroot
chroot /sysroot
passwd root
touch /.autorelabel        # REQUIRED for SELinux, otherwise login fails
exit
exit
```

### Emergency / rescue

- Append `systemd.unit=rescue.target` or `systemd.unit=emergency.target` to the `linux` line.
- Typical cause of emergency mode: **bad `/etc/fstab` entry**. Fix the line (or add `nofail`), then `mount -a` to test.

## 6. Software Management

> ➕ **Added**: this whole section.

### dnf / rpm

```
dnf install|remove|update <pkg>
dnf search keyword ; dnf info pkg
dnf provides */semanage            # which package ships this file?
dnf list installed ; dnf history
dnf group list ; dnf group install "Development Tools"
rpm -qa ; rpm -qi pkg ; rpm -ql pkg ; rpm -qf /path/file
rpm -ivh pkg.rpm
```

### Configure a repository

Create `/etc/yum.repos.d/local.repo`:

```
[BaseOS]
name=BaseOS
baseurl=http://server/BaseOS
enabled=1
gpgcheck=0

[AppStream]
name=AppStream
baseurl=http://server/AppStream
enabled=1
gpgcheck=0
```

```
dnf clean all
dnf repolist
```

Or with a local DVD/ISO: `mount /dev/sr0 /mnt` (add it to `/etc/fstab`), then point `baseurl=file:///mnt/BaseOS`.

### Modules (AppStream streams)

```
dnf module list
dnf module enable nodejs:20
dnf module install nodejs:20
```

# 7. Scheduling Tasks

> ➕ **Added**: syntax details, `at`, timers.

## cron

```
crontab -e                  # edit your own
crontab -eu natasha         # edit another user's (as root)
crontab -lu natasha         # list natasha's jobs
crontab -r                  # remove all
```

Format: `minute hour day-of-month month day-of-week command`

```
30 2 * * 1-5   /usr/local/bin/backup.sh      # 02:30, Mon-Fri
*/10 * * * *   logger "ten minutes"
0 9 1 * *      echo "first of month"
```

System jobs: `/etc/crontab`, `/etc/cron.d/`, `/etc/cron.daily|hourly|weekly|monthly/`.
Access control: `/etc/cron.allow`, `/etc/cron.deny`.

## at (one-off)

```
systemctl enable --now atd
at 17:30          # then type commands, finish with Ctrl+D
atq ; atrm 3
```

## systemd timers

```
systemctl list-timers
```

A timer `foo.timer` triggers `foo.service`; enable with `systemctl enable --now foo.timer`.

---

# 8. Networking & SSH

## NetworkManager (`nmcli`)

```
nmcli con show                    # list connections
nmcli con show enp0s3             # details
nmcli dev status

# create a static connection
nmcli con add con-name New type ethernet ifname enp0s3 \
```

```
  ipv4.method manual ipv4.addresses 172.16.45.1/24 ipv4.gateway
172.16.45.254 \
  ipv4.dns 8.8.8.8

nmcli con mod New +ipv4.dns 1.1.1.1
nmcli con mod New ipv4.method auto          # back to DHCP
nmcli con up New
nmcli con del New
nmtui                                        # text UI alternative
```

> 🖍️ **Fixed**: the command is `nmcli` (not `numcli`), and IPs need a prefix length (`/24`). The original `172.268.x.x` is also not a valid address (octets ≤ 255).

## Hostname & name resolution

```
hostnamectl set-hostname server1.example.com
hostnamectl status
cat /etc/hosts
cat /etc/resolv.conf
```

Useful: `ip a`, `ip r`, `ss -tulpn`, `ping`, `dig host`, `nslookup`.

## SSH: root password login

Connect: `ssh root@192.168.100.189`

Allow root login with a password, in `/etc/ssh/sshd_config.d/01-permitrootlogin.conf`:

```
PermitRootLogin yes
PasswordAuthentication yes
```

```
systemctl restart sshd
sshd -T | grep -Ei "permitrootlogin|passwordauthentication"
```

> Root login is disabled-by-policy on real systems; use only for the lab.

## SSH keys (➕ Added)

```
ssh-keygen -t ed25519
ssh-copy-id user@host
ssh user@host                  # now passwordless
```

Disable password auth later: `PasswordAuthentication no` in sshd config, then restart sshd. If you change the SSH port, also update **SELinux** (`semanage port`) and the **firewall**.

---

## 9. Firewall

> ➕ **Added**: the original notes only had the heading.

`firewalld` uses **zones** and **services**.

```
systemctl enable --now firewalld
firewall-cmd --state
firewall-cmd --get-default-zone ; firewall-cmd --get-active-zones
firewall-cmd --list-all [--zone=public]

# open things (ALWAYS --permanent, then reload)
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --permanent --remove-service=cockpit
firewall-cmd --reload

# source / interface
firewall-cmd --permanent --zone=trusted --add-source=192.168.100.0/24
firewall-cmd --permanent --zone=public --change-interface=enp0s3

# port forward 80 → 8080
firewall-cmd --permanent --add-forward-port=port=80:proto=tcp:toport=8080
```

Tip: `firewall-cmd --get-services` lists valid service names.

---

## 10. SELinux

SELinux = a set of rules defining **which process may access which file, directory or port**.

- **Context** (label) = `user:role:type:level`. The **type** is what the policy matches. Example: files in `/var/www/html` carry `httpd_sys_content_t`, so only Apache can serve them. Moving them elsewhere breaks access.

### Modes

| Mode | Value | Behavior |
|---|---|---|
| Enforcing | 1 | policy enforced |
| Permissive | 0 | violations only logged |
| Disabled | n/a | no SELinux (needs reboot) |

```
getenforce
setenforce 0        # permissive (until reboot)
```

```
setenforce 1        # enforcing
```

Persistent: `/etc/selinux/config` → `SELINUX=enforcing`

## File contexts

```
dnf install -y policycoreutils-python-utils     # provides semanage

ls -lZ file ; ls -ldZ /dir          # view context
ps -eZ | grep httpd                 # process context

# set a persistent rule, then apply it
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
restorecon -Rv /web
```

- `(/.*)?` = the directory itself **and** everything beneath it.
- `chcon` changes a label temporarily; it is lost on relabel. Always prefer `semanage fcontext` + `restorecon`.
- Check what a type is for: `semanage fcontext -l | grep httpd`

## Booleans

```
getsebool -a | grep httpd
setsebool -P httpd_can_network_connect on     # -P = persistent
semanage boolean -l | grep nfs
```

## Ports

```
semanage port -l | grep http
semanage port -a -t http_port_t -p tcp 8080    # let httpd bind 8080/tcp
```

(🖍️ the type is `http_port_t`, not `httpd_port_t`.)

## Troubleshooting (➕ Added)

```
dnf install -y setroubleshoot-server
ausearch -m AVC -ts recent
sealert -a /var/log/audit/audit.log
journalctl -t setroubleshoot
```

Example workflow for Apache serving a custom docroot:

```
semanage fcontext -a -t httpd_sys_content_t "/srv/web(/.*)?"
restorecon -Rv /srv/web
```

```
# change DocumentRoot in /etc/httpd/conf/httpd.conf
systemctl restart httpd
```

---

## 11. Storage: Partitions, Swap, fstab

```
lsblk ; blkid ; df -hT ; findmnt
```

**Why partition?** Boot multiple OSes · separate swap space · organize/isolate data.

### Create a partition

```
fdisk /dev/sdc
#  n  → new   (accept defaults, +size)
#  t  → type  (82 / swap for swap partitions)
#  d  → delete
#  p  → print
#  w  → write and exit
partprobe /dev/sdc        # make kernel re-read the table without reboot
lsblk
```

> 🧨 Run `partprobe` on the **disk** ( `/dev/sdc` ), not the partition.

### Filesystems

| Type | Notes |
|---|---|
| **XFS** | RHEL default, high performance, large files. **Can grow but not shrink.** |
| **ext4** | General purpose, light on CPU/RAM; used on Android. Can grow and shrink. |
| **Btrfs** | Not supported in RHEL. |

```
mkfs.xfs /dev/sdc1
mkfs.ext4 /dev/sdc1
```

### Mount

```
mkdir /mnt/data
mount /dev/sdc1 /mnt/data
umount /mnt/data
```

**Persistent mount**: `/etc/fstab`

```
# device          mount point   fs    options   dump  fsck
/dev/sdc1         /mnt/data     xfs   defaults  0     0
UUID=xxxxxxxx     /mnt/data     xfs   defaults  0     0     # recommended
```

```
blkid /dev/sdc1             # get the UUID
mount -a                    # TEST fstab before rebooting!
systemctl daemon-reload
```

> A wrong fstab line → **emergency mode on boot**. Always run `mount -a` after editing.

### Remove a partition

```
umount /mnt/data            # + remove its fstab line
fdisk /dev/sdc              # d → (number) → w
lsblk
```

### Swap partition

```
fdisk /dev/sdc              # n, t (82 Linux swap), w
partprobe /dev/sdc
mkswap /dev/sdc1
swapon /dev/sdc1
echo "/dev/sdc1 none swap defaults 0 0" >> /etc/fstab
swapon --show ; free -h
```

### Swap file

```
fallocate -l 256M /swapfile        # 🖍️ the original note had a typo (fa::llocate)
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo "/swapfile none swap defaults 0 0" >> /etc/fstab
swapon --show
```

Add `/swapfile` to `/etc/fstab` as above, then `mount -a` / `swapon -a`.

---

## 12. LVM & VDO

**Why LVM?** Combine disks into one pool. Example: 3 × 4 GB disks (= 12 GB) can serve one 11 GB volume, and it can be resized later.

```
Physical disks → PV (pvcreate) → VG (vgcreate) → LV (lvcreate) → filesystem
→ mount
```

## Create

```
# 1. Physical volumes
pvcreate /dev/sdb /dev/sdc /dev/sdd
pvs ; pvdisplay /dev/sdb

# 2. Volume group
vgcreate nameVG /dev/sdb /dev/sdc /dev/sdd
vgs ; vgdisplay nameVG

# 3. Logical volume
lvcreate -n LV1 -L 11G nameVG          # fixed size
lvcreate -n LV2 -l 100%FREE nameVG     # all remaining space
lvs ; lvdisplay /dev/nameVG/LV1

# 4. Filesystem + persistent mount
mkfs.xfs /dev/nameVG/LV1
mkdir /data
echo "/dev/nameVG/LV1 /data xfs defaults 0 0" >> /etc/fstab
mount -a
```

## Custom physical extent size

Default PE = **4 MiB**. Set it when creating the VG:

```
vgcreate -s 8M nameVG2 /dev/sdd
vgdisplay nameVG2          # PE Size: 8 MiB
```

LVs created from this VG use 8 MiB extents. (A small amount of space is reserved for metadata.)

## Extend a logical volume

```
lvextend -r -L +2G /dev/nameVG/LV1        # grow by 2G and resize the FS (-
r)
lvextend -r -L 8G  /dev/nameVG/LV1        # grow TO 8G
lvextend -r -l +100%FREE /dev/nameVG/LV1  # use all free space
```

`-r` resizes the filesystem too. Without it:

```
xfs_growfs /data           # XFS (give the mount point)
resize2fs /dev/nameVG/LV1  # ext4
```

**If the VG has no free space**, extend the VG first:

```
pvcreate /dev/sdg
vgextend nameVG /dev/sdg
vgs                          # confirm new free space
lvextend -r -L +6G /dev/nameVG/LV1
```

## Shrink (ext4 only, ➕ Added)

XFS cannot shrink. For ext4: unmount → `e2fsck -f` → `resize2fs` → `lvreduce`. Or use `lvreduce -r -L 5G`.

## Remove

```
umount /data         # + remove fstab entry
lvremove /dev/vg/lv
vgremove vg
pvremove /dev/sdb /dev/sdc /dev/sdd
```

## VDO: Virtual Data Optimizer

Saves space using **deduplication**, **compression** and **thin provisioning**.

```
dnf install -y lvm2 kmod-kvdo vdo

pvcreate /dev/sdd
vgcreate VG1 /dev/sdd
lvcreate --type vdo --name VDO1 --size 5G --virtualsize 50G VG1
mkfs.xfs -K /dev/VG1/VDO1        # -K skips discard, makes mkfs fast
mkdir /vdo
echo "/dev/VG1/VDO1 /vdo xfs defaults,x-systemd.requires=vdo.service 0 0" >>
/etc/fstab
mount -a
lvs -a ; vdostats --human-readable
```

`--size` = real physical space; `--virtualsize` = logical size shown to users (roughly 2× to 60× the physical size).

## Stratis (➕ Added, optional)

```
dnf install -y stratis-cli stratisd ; systemctl enable --now stratisd
stratis pool create pool1 /dev/sdb
stratis fs create pool1 fs1
mount /dev/stratis/pool1/fs1 /mnt
# fstab needs: x-systemd.requires=stratisd.service
```

## 13. NFS & autofs

### NFS server

```
dnf install -y nfs-utils
mkdir /sharedfolder
```

`/etc/exports`:

```
/sharedfolder  192.168.100.0/24(rw,sync)
```

```
firewall-cmd --permanent --add-service={nfs,rpc-bind,mountd}   # 🧨 these are *services*, not ports
firewall-cmd --reload

exportfs -rav
systemctl enable --now nfs-server
setsebool -P nfs_export_all_rw on        # when exporting rw with SELinux enforcing
```

### NFS client: manual mount

```
dnf install -y nfs-utils
showmount -e serverIP
mount -t nfs serverIP:/sharedfolder /mnt/nfs
# persistent:
echo "serverIP:/sharedfolder /mnt/nfs nfs defaults,_netdev 0 0" >> /etc/fstab
```

### autofs: mount on demand

```
dnf install -y nfs-utils autofs
systemctl enable --now autofs rpcbind
```

`/etc/auto.master`:

```
/auto_mount  /etc/auto.misc
```

`/etc/auto.misc`:

```
access  -rw,soft,intr  serverIP:/sharedfolder
```

```
systemctl restart autofs
cd /auto_mount/access ; ls       # mounts the first time you enter it
```

**Wildcard home directories** (common exam task):

```
# /etc/auto.master.d/home.autofs
/home/remote  /etc/auto.home
# /etc/auto.home
*  -rw,sync  serverIP:/home/remote/&
```

---

## 14. Time Sync (chrony)

```
dnf install -y chrony             # 🖍️ package name is "chrony", the
service is "chronyd"
```

`/etc/chrony.conf` : comment out the default `pool` line and add:

```
server ntp02.oal.ul.pt iburst
server pool.ntp.org iburst
```

> 🖍️ The correct keyword is `iburst` (not `ibrust` ).

```
systemctl enable --now chronyd
systemctl restart chronyd
chronyc sources -v
chronyc tracking

timedatectl                          # ➕ Added
timedatectl set-timezone Africa/Casablanca
timedatectl set-ntp true
```

---

## 15. Logging & journald

### Persistent journal

```
mkdir -p /var/log/journal
chown root:systemd-journal /var/log/journal
chmod 2755 /var/log/journal
systemd-tmpfiles --create --prefix /var/log/journal
```

Edit `/etc/systemd/journald.conf` (🖍️ file name is `journald.conf`):

```
[Journal]
Storage=persistent
```

```
systemctl restart systemd-journald
journalctl --flush
ls /var/log/journal              # verify
```

## Reading the journal (➕ Added)

```
journalctl -u sshd               # one unit
journalctl -b ; journalctl -b -1 # this boot / previous boot
journalctl -p err                # priority
journalctl --since "1 hour ago"
journalctl -f                    # follow
journalctl _UID=1000
```

## rsyslog: custom log rules

```
dnf install -y rsyslog
```

`/etc/rsyslog.d/name.conf`:

```
mail.*    /var/log/journalname-exam.log
```

```
systemctl restart rsyslog
logger -p mail.info "information"
tail /var/log/journalname-exam.log
```

## logrotate (➕ Added)

Configs in `/etc/logrotate.conf` and `/etc/logrotate.d/`:

```
/var/log/myapp.log {
    weekly
    rotate 4
    compress
    missingok
}
```

---

# 16. Containers (Podman)

## VMs vs containers

| | **Virtual machine (VirtualBox, KVM, VMware)** | **Container (Podman, Docker)** |
|---|---|---|
| What it is | A hypervisor emulates a whole computer | A normal host process isolated by the kernel |
| Kernel | Own kernel + OS | **Shares the host kernel** |
| Isolation | Strong | Weaker (a kernel flaw affects all containers) |
| Weight | GBs, ~1 min boot, reserved RAM | MBs, ~1 s start |
| Isolation tech | Virtual hardware | **namespaces** (what it sees) + **cgroups** (CPU/RAM limits) |
| Different OS than host? | Yes | No (Linux containers need a Linux kernel) |

## Images & containers

```
dnf install -y container-tools
podman login registry.example.com
podman search httpd ; podman pull registry/image:tag
skopeo inspect docker://registry/image:tag
podman images ; podman ps -a
podman run -d --name web -p 8080:80 -e VAR=val image
podman exec -it web /bin/bash
podman logs web
podman stop|start|rm web ; podman rmi image
```

Registries are configured in `/etc/containers/registries.conf`.

## Build an image

Inside a directory with a `Containerfile`:

```
podman build -t example .
podman image build -t web_image -f http://server/Containerfile    # from a
URL
```

## Volumes & SELinux labels

```
podman run -d -v /host/dir:/container/dir:Z --name example example
```

> 🖍️ **Fixed**: `:Z` (uppercase) relabels the volume as **private to this one container**. `:z` (lowercase) relabels it as **shared** between several containers.

Port publishing needs the port allowed in the firewall (and SELinux if non-standard).

### Run as a systemd service: rootless (the exam way)

```
# as root, once: allow the user's services to run without an active login
loginctl enable-linger username      # 🖍️ "enable-linger" (hyphen)

# then log in as the user directly (ssh or `su - username`, NOT `su` without
dash)
ssh username@localhost
mkdir -p ~/.config/systemd/user
cd ~/.config/systemd/user
podman generate systemd --name example --files --new
#   → creates container-example.service  ([Service] … [Install]
WantedBy=default.target)
systemctl --user daemon-reload
systemctl --user enable --now container-example.service
systemctl --user status container-example.service
```

### Run as a systemd service: root

```
cd /etc/systemd/system
podman generate systemd --name example --files --new
systemctl daemon-reload
systemctl enable --now container-example.service
```

> Note: `podman generate systemd` is deprecated upstream in favour of Quadlet, but it still works on RHEL 9 and is what the exam expects.

---

## 17. Shell Scripting

> ➕ **Added**: RHCSA requires basic scripting.

```
#!/bin/bash
# Arguments: $0 name, $1..$9, $@ all, $# count, $? last exit status

if [ $# -ne 1 ]; then
    echo "Usage: $0 username" >&2
    exit 1
```

```
fi

user="$1"

if id "$user" &>/dev/null; then
    echo "$user exists"
else
    useradd "$user"
    echo "NewPassword" | passwd --stdin "$user"
fi

for f in /etc/*.conf; do
    [ -f "$f" ] && echo "$f"
done

while read -r line; do
    echo "$line"
done < /etc/hosts
```

Make it executable: `chmod +x script.sh`. Test operators: `-f` file, `-d` dir, `-z` empty string, `-eq -ne -gt -lt` numbers, `==` strings.

Typical exam task: a script that takes input and creates users/finds files. See `find` examples in §1.

---

# 18. Exam Tips & Final Checklist

## Habits that save you

- **Reboot at the end** and confirm everything survives (mounts, services, SELinux, firewall).
- After editing `/etc/fstab` → `mount -a` before rebooting.
- Always `--permanent` + `--reload` with firewalld.
- Always `systemctl enable --now`, not just `start`.
- SELinux: use `semanage` + `restorecon`, never rely on `chcon`.
- Need a file/option name? Use `man -k`, `dnf provides`, and `/usr/share/doc`.
- Never disable SELinux or the firewall to "fix" a task.

## Quick checklist by objective

- [ ] Redirection, grep, regex, find, tar, ssh, scp, links
- [ ] Users, groups, password aging, sudo, umask
- [ ] chmod, SUID/SGID/sticky, ACLs
- [ ] systemctl, targets, tuned, nice/renice, kill

- [ ] GRUB edit, root password reset ( `rd.break` ), emergency mode
- [ ] dnf repos, rpm queries, modules
- [ ] cron, at
- [ ] nmcli static IP, hostname, `/etc/hosts`
- [ ] firewall-cmd services/ports
- [ ] SELinux modes, contexts, booleans, ports, troubleshooting
- [ ] Partitions, fstab, UUIDs, swap
- [ ] LVM create/extend/remove, VDO
- [ ] NFS server/client, autofs (incl. wildcard)
- [ ] chrony, timedatectl
- [ ] journald persistence, rsyslog, logrotate
- [ ] Podman: pull/run/build, volumes ( `:Z` ), systemd user service, linger
- [ ] Basic bash scripts